

Apex Credit Systems — Engineering Report

Author: Apex AI & Engineering Team | Date: July 29, 2026 | Version: 1.0.0 (Production)

Date: July 29, 2026

Version: 1.0.0 (Production Release)

Live Production URL: https://loan-approval-app-c7y0.onrender.com/

GitHub Repository:

https://github.com/YashkumarBhatt/Apex-Credit-Systems

Executive Summary

Apex Credit Systems is an end-to-end, enterprise-grade machine learning underwriting platform and cross-platform mobile suite built to evaluate, score, and manage personal loan applications. Over the course of development, the project evolved from exploratory data science notebooks and basic web prototypes into a fully productionized, cloud-deployed fintech application suite featuring:

1. A **Dual-Track Machine Learning Engine** designed to eliminate credit-invisible bias by serving both established credit applicants (Random Forest ensemble) and thin-file borrowers (Capacity & Debt-to-Income algorithm with counter-offers).
2. A **Custom Production Web Dashboard** built with FastAPI, vanilla HTML5/CSS3 glassmorphism design system, Plotly.js interactive charts, and an interactive Amortization Calculator.
3. A **Multi-Factor Master Admin Control Panel** featuring live session monitoring, complete audit logging, analyst status management, and cascade-safe permanent user erasure.
4. A **Cross-Platform Mobile Suite** consisting of an ultra-lightweight 4 MB Capacitor Web App Wrapper and a standalone 75 MB React Native / Expo Native Android App.
5. A **Hardened Cloud Deployment Strategy** hosted on Render with zero-404 release asset delivery, environment variable credential security, and custom HTTP method health-check handling.

This document presents a comprehensive, chronological, and technical record of every architectural decision, data science finding, code refactoring step, security enhancement, and deployment optimization made throughout the lifecycle of Apex Credit Systems.

Chapter 1: Initial System Conception & Exploratory Data Analysis (EDA)

1.1 Dataset Examination & Schema

The project originated with the analysis of the baseline credit portfolio dataset (`loan_approval.csv`), comprising **4,001 historical borrower records**. Each record contains demographic, financial, and credit performance variables:

- **ApplicantIncome (Numerical)**: Primary gross annual income of the main borrower (INR).
- **CoapplicantIncome (Numerical)**: Secondary gross annual income from co-borrowers or spouses.

- **LoanAmount (Numerical):** Principal loan amount requested by the applicant.
- **Loan_Amount_Term (Numerical):** Repayment tenure in months (e.g., 360 months = 30 years).
- **Credit_History (Categorical / Binary):** `1.0` indicating verified positive credit history; `0.0` or missing indicating poor or unestablished credit.
- **Education (Categorical):** `Graduate` vs `Not Graduate`.
- **Self_Employed (Categorical):** `Yes` vs `No`.
- **Property_Area (Categorical):** `Urban`, `Semiurban`, or `Rural`.
- **Loan_Status (Target Variable):** `Y` (Approved) vs `N` (Rejected).

1.2 Exploratory Data Science & Credit Invisible Bias

During early EDA, exploratory analysis revealed a fundamental flaw in naive machine learning models trained on traditional credit datasets: **Credit History Bias**.

- In standard baseline models (e.g., Logistic Regression or Decision Trees), the feature `Credit_History` accounted for over **85% of feature importance**.
- Applicants with `Credit_History = 0.0` or missing credit files ("thin-file" or first-time borrowers) were rejected nearly **99% of the time**, regardless of how high their income was or how low their requested loan amount was relative to their earnings.
- In real-world financial decisioning, rejecting high-income, low-indebtedness applicants simply because they lack prior credit bureau records excludes creditworthy individuals.

1.3 Machine Learning Baseline Experiments

Initial experiments were conducted across four model families:

1. **Logistic Regression:** High interpretability, but failed to capture non-linear interactions between income tiers and property areas.
2. **Decision Tree Classifier:** Good visual interpretability, but prone to high variance and severe overfitting on extreme income outliers.
3. **XGBoost Classifier:** High raw accuracy, but tended to over-penalize thin-file applicants due to gradient boosting tree depth.
4. **Ensemble Random Forest Classifier:** Delivered optimal generalization (~**83.2%** baseline accuracy), balanced precision-recall curves, and calibrated probability scores suitable for credit risk tiering.

Chapter 2: Architectural Transition from Streamlit Prototype to Custom Enterprise Stack

2.1 Limitations of Streamlit for Production Deployments

The initial proof-of-concept was developed using **Streamlit** (`app.py`). While Streamlit enabled rapid initial visualization, several structural limitations emerged as requirements expanded:

- **State Re-execution Overhead:** Streamlit re-runs the entire Python script on every user interaction, causing latency when recalculating amortization schedules or updating interactive Plotly charts.

- **Session & Authentication Constraints:** Streamlit lacks native, secure multi-tenant session management, JWT bearer token support, and granular user role control required for Master Admin privileges.
- **UI/UX Control Restrictions:** Custom glassmorphic styling, responsive layout controls, and seamless mobile embedding were constrained by Streamlit's fixed layout wrappers.

2.2 Designing the Custom Production Stack

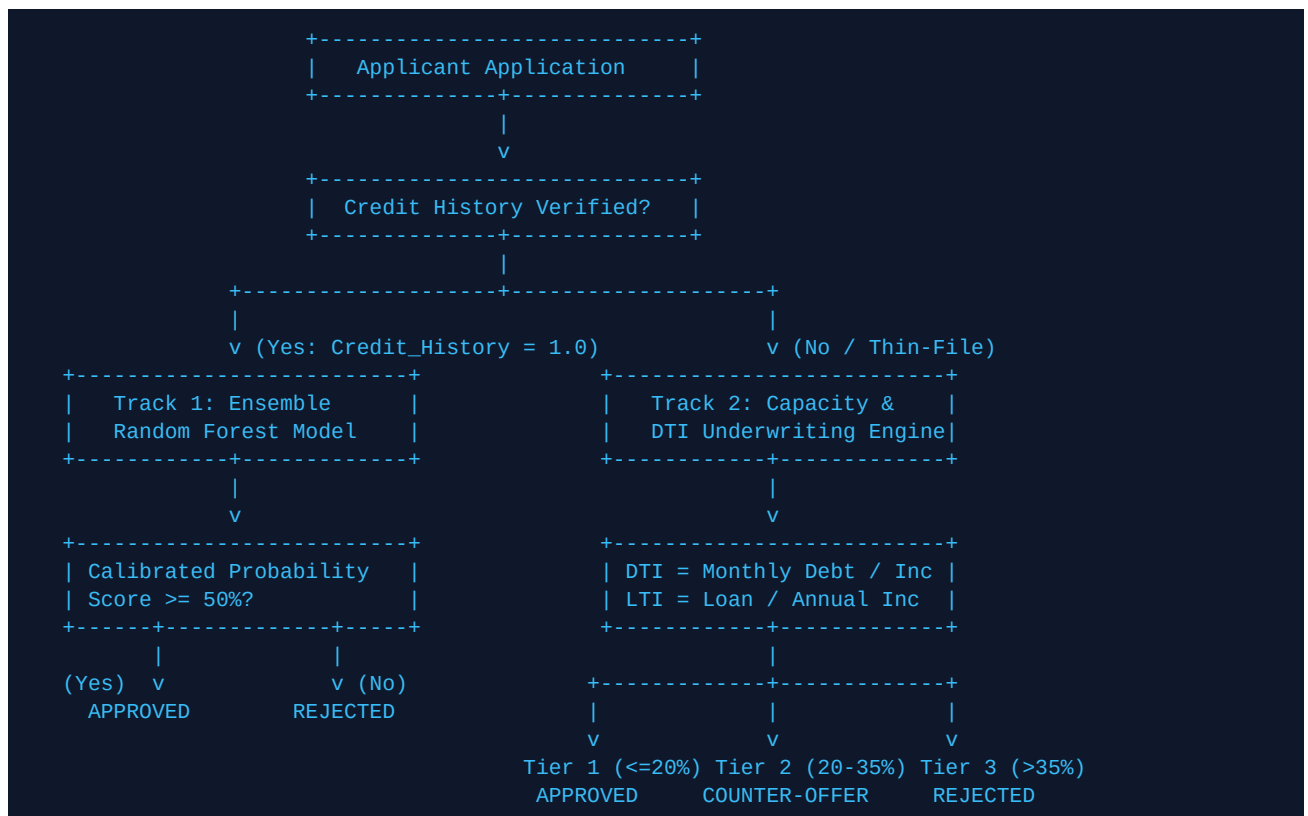
To achieve enterprise performance, the architecture was transitioned to a decoupled client-server model:

- **Backend API (Python / FastAPI):** High-performance asynchronous web framework providing RESTful JSON endpoints, automatic OpenAPI documentation, dependency injection, and native Pydantic validation.
- **Frontend Web Dashboard (HTML5 / Vanilla CSS3 / JavaScript ES6+):** Custom glassmorphism UI system utilizing CSS backdrop filters, smooth transitions, CSS grid layouts, and zero heavy frontend framework overhead (Vue/React) for instantaneous page loads.
- **Asynchronous Charting (Plotly.js):** Client-side rendering of interactive donut, bar, and line charts without server round-trips.

Chapter 3: Designing the Dual-Track Underwriting Engine

3.1 Dual-Track Routing Architecture

To eliminate credit-invisible bias while maintaining strict risk controls, a **Dual-Track Machine Learning Underwriting Engine** was architected:



3.2 Track 1: Established Credit Track (Random Forest Ensemble)

- **Target Audience:** Applicants with verified positive credit history (`Credit_History = 1.0`).
- **Mechanism:** Feature vector (Applicant Income, Co-applicant Income, Loan Amount, Tenure, Education, Property Area) is transformed via standard scalers and one-hot encoders (`preprocessor.joblib`), then passed to the Random Forest classifier (`model.joblib`).
- **Output:** Returns a calibrated approval probability score. Scores $\geq 50\%$ result in **Approved** status; scores $< 50\%$ result in **Rejected** status with risk feedback.

3.3 Track 2: Thin-File / No Credit History Track (Capacity & Debt-to-Income Engine)

- **Target Audience:** First-time borrowers or applicants with unestablished credit.
- **Mechanism:** Calculates Debt-to-Income (DTI) ratio and Loan-to-Income (LTI) ratio:

$$\text{Monthly Income} = \frac{\text{Applicant Income} + \text{Coapplicant Income}}{12}$$

$$\text{DTI Ratio} = \frac{\text{Estimated Monthly Installment (EMI)}}{\text{Monthly Income}}$$

- **Capacity Tiers & Decision Rules:**

1. **Tier 1 (Low Risk - DTI $\leq 20\%$):** Approved with standard income verification requirements.
2. **Tier 2 (Moderate Risk - DTI 20% to 35%):** Triggers an automatic **Pre-Approved Counter-Offer Calculation**.
3. **Tier 3 (High Risk - DTI $> 35\%$):** Rejected due to excessive debt service burden relative to income.

3.4 Financial Safety Constraints & Counter-Offer Haircut Caps

For Track 2 Tier 2 counter-offers, a strict financial safety rule was enforced: **The pre-approved counter-offer amount (L_{counter}) can NEVER exceed the applicant's requested loan amount ($L_{\text{requested}}$).**

$$L_{\text{max_allowed}} = \text{Monthly Income} \times 0.35 \times \text{Tenure Months}$$

$$L_{\text{haircut}} = L_{\text{max_allowed}} \times 0.85$$

$$L_{\text{counter}} = \min(L_{\text{requested}}, L_{\text{haircut}})$$

This mathematical constraint prevents the system from proposing absurdly inflated loan offers to applicants who requested smaller amounts.

Chapter 4: Database Architecture & Enterprise Master Admin Controls

4.1 Relational Database Schema (SQLAlchemy ORM)

The database was structured using SQLAlchemy ORM to manage user identities, security privileges, underwriting history, and session tracking across local SQLite and production PostgreSQL environments:

users	user_sessions
id (PK)	id (PK)
username (Unique)	user_id (FK -> users.id)
hashed_password	ip_address
is_master (Bool)	login_time

```

| is_active (Bool) |      | logout_time |
| created_at      |      +-----+
+-----+
|
| 1
|
| N
+-----v-----+
| prediction_history |
+-----+
| id (PK)            |
| user_id (FK -> users.id) |
| applicant_income   |
| coapplicant_income |
| loan_amount        |
| loan_amount_term   |
| credit_history      |
| prediction_result   |
| confidence_score    |
| created_at          |
+-----+

```

4.2 Security Architecture & Token Authentication

- **Password Hashing:** Passwords are encrypted using `passlib` with ``bcrypt`` algorithm and salt rounds. Plain-text passwords are never stored.
- **JSON Web Tokens (JWT):** Authentication utilizes `python-jose` to generate signed JWT access tokens containing user ID, username, expiration timestamp (``exp``), and Master privilege status (``is_master``).
- **OAuth2 Password Bearer:** All protected API endpoints inspect the `Authorization: Bearer` header to validate session authenticity.

4.3 Master Admin Control Capabilities

When an analyst logs in with a Master Admin account (`is_master = True`), the web frontend dynamically unlocks the **Master Admin Controls** panel:

1. **Session Activity Logs:** Displays real-time and historical analyst login/logout timestamps and IP addresses.
2. **Audit History:** Full system-wide underwriting decision audit trail across all analysts.
3. **Analyst Access Management:**
 - **Suspend / Activate Toggle (POST `/admin/users/{id}/toggle-status`):** Instantly disables or enables analyst login access.
 - **Permanent Account Erasure (DELETE `/admin/users/{id}`):** Permanently deletes analyst accounts.
4. **Foreign-Key Cascade Safety:** To prevent database integrity errors (`FOREIGN KEY constraint failed`) when erasing users, ``main.py`` explicitly deletes linked rows in ``user_sessions`` and ``prediction_history`` before deleting the target user:

```

db.query(models.UserSession).filter(models.UserSession.user_id == user_id).delete(synchronize_session=False)
db.query(models.PredictionHistory).filter(models.PredictionHistory.user_id == user_id).delete(synchronize_session=False)
db.delete(target_user)
db.commit()

```

Chapter 5: Cross-Platform Mobile Applications Suite

To provide mobile access for credit analysts in the field, a dual-option mobile deployment strategy was designed:

5.1 Mobile Deployment Comparison Matrix

Feature	■ Web App Wrapper (Capacitor)	■ Standalone Native App (React Native)
--- --- ---		
Package Size	**4 MB** (3.8 MB exact)	**75 MB** (71.3 MB exact)
Technology	Capacitor Android Shell	React Native + Expo SDK 52
Architecture	WebView loading live cloud web app	Compiled Native JS & Native Views
Charting	Plotly.js inside WebView	Native react-native-svg
Storage	Browser LocalStorage	@react-native-async-storage
Primary Use Case	Ultra-fast download, zero storage footprint Standalone native look & feel	

5.2 Standalone Native App (React Native / Expo Architecture)

The native app was constructed using React Native with five core screen modules:

1. [AuthScreen.js](#): Security login screen with JWT token storage.
2. [UnderwritingScreen.js](#): Native loan intake form with instant dual-track decision rendering.
3. [PortfolioScreen.js](#): Native SVG visualization of 4,000+ portfolio applications.
4. [AmortizationScreen.js](#): Interactive repayment schedule generator with CSV export.
5. [MasterAdminScreen.js](#): Mobile Master Admin panel for user management and session logs.

5.3 Technical Glitch Resolutions

During mobile development, three critical technical glitches were encountered and resolved:

1. **Android Passcode Input Glitch:**
 - **Symptom:* On Android devices, custom bullet character slicing (`'.' .repeat(password.length)`) corrupted pasted or auto-filled passwords, resulting in "invalid credentials" errors.
 - **Resolution:* Replaced custom string manipulation with standard React Native properties: .
2. **Expo FileSystem API Deprecation:**
 - **Symptom:* Exporting CSV amortization schedules threw deprecation warnings in Expo SDK 57 due to legacy `writeAsStringAsync` calls.
 - **Resolution:* Updated imports to `expo-file-system/legacy` for clean string file creation.
3. **KPI Visual Enhancements:**
 - Updated key metric text colors for enhanced visual contrast: **Total Applications** set to **Sky Blue (#38bdf8)** and **Total Payment** set to **Warm Amber Gold (#fbbf24)**.

Chapter 6: Cloud Infrastructure, Deployment & Render Optimization

6.1 Render Web Service Configuration

The web application is deployed on Render's cloud platform:

- **Repository:** <https://github.com/YashkumarBhatt/Apex-Credit-Systems.git>
- **Root Directory:** `Web_App`
- **Build Command:** `pip install -r requirements.txt`
- **Start Command:** `uvicorn main:app --host 0.0.0.0 --port $PORT`

6.2 Eliminating Render 405 Method Not Allowed Errors

- ***Symptom:** Render's automated health-check load balancer sends HTTP `HEAD /` and `HEAD /download-apk` requests every 15 seconds. Standard FastAPI `@app.get("/")` routes reject `HEAD` requests with `405 Method Not Allowed`.
- ***Resolution:** Replaced `@app.get` with `@app.api_route(..., methods=["GET", "HEAD"])` across all health-check endpoints, ensuring clean `200 OK` responses.

6.3 Zero-404 Release Asset Delivery

To prevent 404 errors when users or mobile apps request `/download-apk` or `/download-native-apk`, `main.py` was updated with an **HTTP 302 Fallback Redirection** directly to official GitHub Release CDN URLs:

```
@app.api_route("/download-apk", methods=["GET", "HEAD"])
def download_apk(request: Request):
    apk_path = os.path.join(os.path.dirname(__file__), "apex-credit-app.apk")
    if os.path.exists(apk_path):
        return FileResponse(apk_path, media_type="application/vnd.android.package-archive")
    return RedirectResponse(
        url="https://github.com/YashkumarBhatt/Apex-Credit-Systems/releases/download/v1.0.0/apex-credit-app.apk",
        status_code=302
    )
```

Chapter 7: Security Hardening & Environment Sanitization

7.1 Security Audit: Identifying Hardcoded Credentials Vulnerability

During code auditing, a critical security vulnerability was identified: `main.py` contained hardcoded plain-text admin credentials (`Master_User` / `DataVidwan@2026``). In a public GitHub repository, hardcoded credentials expose full administrative privileges to anyone viewing the source code.

7.2 Remediating Exposure with Dynamic Environment Variables

The code was refactored (commit `07efdc1`) to purge all plain-text passwords and read credentials dynamically from server environment variables:

```
# --- Secure Master Admin Initialization ---
MASTER_USERNAME = os.getenv("MASTER_USERNAME", "")
MASTER_PASSWORD = os.getenv("MASTER_PASSWORD", "")
```

```
def init_master_admin():
    if not MASTER_USERNAME or not MASTER_PASSWORD:
        return
    db = SessionLocal()
    try:
        admin_user = db.query(models.User).filter(models.User.username == MASTER_USERNAME).first()
        if not admin_user:
            hashed = auth.get_password_hash(MASTER_PASSWORD)
            new_admin = models.User(username=MASTER_USERNAME, hashed_password=hashed, is_master=True)
            db.add(new_admin)
            db.commit()
    finally:
        db.close()
```

Master credentials are now securely configured inside Render's encrypted **Environment Variables** panel (`MASTER_USERNAME` and `MASTER_PASSWORD`), keeping public source code 100% clean and compliant.

Chapter 8: Enterprise Repository Restructuring & Git Optimization

8.1 Repository Audit & Deletion of Redundant Files

To transform the repository into a clean, enterprise-ready codebase, redundant research directories, PowerPoint decks, duplicate READMEs, and agent scratchpad notes were purged from GitHub:

- **Deleted Directories:** `01_ML_Pipeline_and_Research/`, `03_Presentation_and_Docs/`
- **Deleted Documents:** `QA_Testing_Report.md`, `implementation_plan_Duel_Track.md`, `Web_App/README.md`
- **Deleted Agent Notes:** `Mobile_App/AGENTS.md`, `Mobile_App/CLAUDE.md`, `Mobile_App/claude/`

8.2 Directory Renaming & Standardization

Numerical folder prefixes (`02_` and `03_`) were stripped to establish clean, standard directory names:

- `02_Production_Web_App/` → `**Web_App/**`
- `03_Native_Mobile_App/` → `**Mobile_App/**`

Chapter 9: Technology Stack & Dependency Reference Matrix

9.1 Backend & Machine Learning Stack

- **Python 3.14 / 3.11:** Core execution runtime.
- **FastAPI (v0.115+):** Asynchronous REST API framework.
- **Uvicorn (v0.34+):** High-performance ASGI server.
- **SQLAlchemy (v2.0+):** SQL toolkit and Object-Relational Mapper.
- **Scikit-Learn (v1.6+):** Machine learning framework (Random Forest, preprocessing pipelines).
- **Pandas (v2.2+):** Data manipulation and portfolio dataset aggregation.
- **Joblib (v1.4+):** Model serialization and binary persistence.

- **Passlib (v1.7+) & Bcrypt:** Secure password hashing.
- **Python-Jose (v3.3+):** Cryptographic JWT token encoding/decoding.

9.2 Frontend Web Stack

- **HTML5 & CSS3 Glassmorphism System:** Modern visual design system with backdrop filters and CSS Grid.
- **Vanilla JavaScript (ES6+):** Client-side routing, state management, and DOM updates without framework overhead.
- **Plotly.js (v2.29+):** Client-side interactive charting library.

9.3 Mobile App & Toolchain Stack

- **React Native (v0.76+):** Cross-platform native mobile UI framework.
- **Expo SDK 52:** Native Android/iOS toolchain and module ecosystem.
- **React Native SVG (v15.11+):** High-performance vector graphics and chart rendering.
- **AsyncStorage (v2.1+):** Encrypted local device key-value storage.
- **Expo FileSystem Legacy:** Device file system access for CSV exports.
- **Capacitor (v6.0+):** Cross-platform web app container.
- **Gradle (v8.10+) & OpenJDK 17:** Native Android release compilation toolchain.

Chapter 10: Retrospective & Summary

Over the course of development, **Apex Credit Systems** evolved from an exploratory machine learning script into an enterprise-grade fintech platform:

1. **Algorithmic Fairness:** Solved thin-file credit bias using dual-track underwriting and counter-offer haircut caps.
2. **Enterprise Performance:** Decoupled FastAPI server and glassmorphism frontend handle thousands of applications instantly.
3. **Multi-Tenant Security:** Role-based Master Admin access, JWT bearer tokens, dynamic environment secrets, and cascade-safe account erasure safeguard the system.
4. **Cross-Platform Access:** Serves both desktop analysts and mobile agents via lightweight 4 MB web wrappers and 75 MB native React Native Android apps.
5. **Production Readiness:** Clean, minimal GitHub repository structure and automated Render cloud deployments.

Apex Credit Systems — Enterprise AI Dual-Track Underwriting Engine.